The 6 Dynamic Workflow Patterns

The copy-paste prompts behind every dynamic workflow in Claude Code

Companion to the video: Master Dynamic Workflows

by Mark Kashef · skool.com/earlyaidopters/about

How to use this

- Pick the pattern that matches your task (each page says when to use it).
- Paste the prompt into Claude Code and swap the folders and details for your own.
- The trigger words are "build a workflow" or "use a workflow" (or just the word "ultracode").
- Confirm the planned workflow when Claude shows it, then let it run.
- Cap the cost any time by adding "use 10k tokens" right in the prompt.

The one idea behind all six

A dynamic workflow spins up a team of separate Claudes instead of doing everything in one context window. That fixes the three ways a single window breaks down on a big job:

- Laziness, it stops early and declares a multi-part job done after partial progress.
- Self-preference, it is biased toward its own answers when asked to check its own work.
- Goal drift, it loses edge cases and "do not do X" rules as it compresses over many turns.

The six patterns are the whole alphabet. Almost every workflow is one of these, or a few stacked together. In build order, simplest first: classify and act, fan out and synthesize, adversarial verification, generate and filter, tournament, loop until done.

PATTERN 1 · Classify and act

WHEN TO USE

- A flood of mixed items that each need the right specialist
- Support queues, inboxes, incoming requests, routing tasks to the right model

THE PROMPT

```
Build a workflow that triages my support inbox in ./support_tickets by spawning
a classifier agent that reads each ticket and routes it to a bug, refund, lead,
or spam handler, deduping against what is already tracked before any handler
acts. Quarantine the reading agents so the ones touching the untrusted ticket
text can only classify and summarize, never take actions, and hand everything
off to a separate trusted agent that does the actual routing and replies. Pair
it with /loop so it keeps clearing the queue as new tickets land.
```

WHAT IT DOES

- A classifier reads each item and routes it to the right handler
- Quarantine bars the agents reading untrusted text from taking actions
- Pair with /loop to keep clearing the queue

PATTERN 2 · Fan out and synthesize

WHEN TO USE

- One big job that splits into many independent pieces
- Deep research, due diligence, reading a whole pile of files at once

THE PROMPT

```
Build a workflow that does due diligence on the data room in ./data_room by
fanning out one subagent per folder, each in its own clean context so the files
never cross contaminate, and having every agent return a structured summary
with the exact source path for each finding. Then run a barrier synthesize step
that waits for all of them to finish and merges their outputs into one cited
diligence memo at ./diligence_memo.md, where every claim links back to the file
it came from.
```

WHAT IT DOES

- One agent per piece, each in a clean context so they do not contaminate each other
- A barrier synthesize step waits for all of them, then merges into one cited result

PATTERN 3 · Adversarial verification

WHEN TO USE

- You need something genuinely checked, not graded by the agent that made it
- Claims, reports, pitch decks, audits, rule adherence

THE PROMPT

```
Use a workflow to go through my blog post draft in ./draft.md and verify every
factual and technical claim before I ship it. Have one agent extract each claim
into its own item, then for every claim spin off a separate agent that checks
it against the real source in ./codebase and flags any claim that does not hold
up. The agent that verifies a claim must be a different agent from the one that
pulled it so it is not just rubber-stamping its own work. When it is done, give
me back the list of claims that failed and the exact reason each one failed so
I know what to fix before publishing.
```

WHAT IT DOES

- One agent extracts every claim, a separate skeptic verifies each against the source
- Defeats self-preferential bias, nothing approves its own work

PATTERN 4 · Generate and filter

WHEN TO USE

- Taste-based work where you want the best of many options
- Names, titles, headline angles, designs, cold-email copy

THE PROMPT

```
Use a workflow to brainstorm 40 video title and headline angle options for the
topic in ./topic.md with one generator agent, then hand them all to a separate
judge agent that scores every option against a rubric for hook strength,
clarity, and curiosity, dedupes the near identical phrasings, and returns only
the top 5 winners with a one line reason for each. The generator that
brainstorms and the judge that scores must be different agents so the judge is
not grading its own ideas.
```

WHAT IT DOES

- A generator makes far more than you need, fast and loose
- A separate judge scores against a rubric, dedupes, and keeps only the best few

PATTERN 5 · Tournament

WHEN TO USE

- Ranking a big pile by judgment where a 1 to 10 score would be unreliable
- Resumes, support tickets by severity, inbound leads by fit

THE PROMPT

```
Use a workflow to rank every resume in ./resumes for the backend engineer role
by running a tournament of pairwise comparisons against a rubric instead of
scoring each one cold, where each head to head match is its own comparison
agent and the deterministic loop holds the bracket so only the running order
stays in context. Once the bracket settles, spin off fresh agents to
double-check the top ten against that same rubric and flag anyone who ranked
higher than they should have. First interview me with the AskUserQuestion tool
to build the rubric before any comparing starts.
```

WHAT IT DOES

- Agents compare two at a time, pairwise judgment beats scoring out of ten
- A deterministic bracket holds the order, a fresh agent runs each matchup

PATTERN 6 · Loop until done

WHEN TO USE

- An unknown amount of work, where you do not know how many passes it will take
- Flaky tests, cleanups, mining sessions for recurring mistakes

THE PROMPT

```
Build a workflow that hunts down a flaky test in ./tests that fails maybe 1 in
50 runs. Keep forming theories about the cause and adversarially testing each
one in its own isolated worktree, looping and spawning new attempts with no
fixed pass count, until the stop condition is met where one theory reproduces
the failure on demand and its fix makes the test pass clean. /goal do not stop
until one theory works and the test is green.
```

WHAT IT DOES

- Keeps spawning until a real stop condition, not a fixed number of passes
- Pair with /goal to set a hard completion requirement

BONUS · Stack them (the composition capstone)

WHEN TO USE

- Real work usually chains a few patterns into one machine. This one is fan out plus adversarial verify plus loop until done.

THE PROMPT

```
Build a workflow that audits every file under ./codebase, fans out one agent
per file, has a separate agent try to refute each finding against the code, and
loops until a clean pass turns up no new issues. Return only the confirmed
issues, each with the file and the exact line. /goal do not stop until a full
clean pass finds no new issues.
```

WHAT IT DOES

- Three patterns stacked into one workflow
- Confirmed issues with almost no false alarms

The control dials

- /goal sets a hard completion requirement. It does not stop until what you asked for is actually true.
- /loop runs a workflow on a schedule. Good for repeatable things like triage every morning.
- Token cap. Add "use 10k tokens" in the prompt and it will not blow past that.

Build your own

You do not have to design the JavaScript by hand. Describe the shape you want in plain English using the pattern words, and Claude writes the harness for you. For example:

```
Use a workflow to <do X over ./your_folder>. Fan out one agent per item, have a
separate agent verify each result, and loop until a clean pass finds nothing
new. use 10k tokens.
```

Want to build these from scratch?

Inside the Early AI Dopters community you get the full living Claude Code course:

- The six workflow patterns taught from zero, with the JavaScript building blocks behind each one.
- The exact prompts and harnesses I use, walked through line by line in a dedicated module.
- Direct access to me and my team of coaches when you hit a wall.
- A working group of builders shipping their own workflows.

Join here: skool.com/earlyaidopters/about